

Coupon Platform 포트폴리오

Coupon Platform

여러 게임이 하나의 플랫폼을 공유해 쓰는 멀티테넌트 쿠폰 발급/사용 서버

GitHub: https://github.com/trisakion0500/coupon_platform

Part 1. 프로젝트 소개

왜 만들었나

게임마다 이벤트 쿠폰(신규가입 보상, 프로모션 코드 등)을 발급하고 검증하는 기능이 필요하다. 문제는 이 기능을 게임마다 각자 구현하면 같은 실수가 반복된다는 점이다.

- 같은 쿠폰 코드가 동시에 두 번 소모되는 레이스 컨디션
- 한정 수량 쿠폰이 목표 수량을 넘겨 발급/소모되는 오버셀
- 게임서버↔쿠폰서버 간 통신을 가로채 재전송하는 리플레이 공격
- 여러 게임(테넌트)이 뒤섞여 서로의 쿠폰 코드/사용 이력을 침범하는 데이터 격리 실패

Coupon Platform은 이 문제를 여러 게임이 공유하는 하나의 플랫폼으로 풀기 위해 설계했다. 게임서버는 쿠폰 발급/사용 시점에 이 서버를 호출하기만 하면 되고, 동시성·보안·테넌트 격리는 플랫폼이 구조적으로 보장한다.

핵심 아이디어

쿠폰 사용은 게임서버가 "이 코드 쓸게" → "지급 결과 알려줄게" 두 단계로 호출하는 게 자연스럽다. 하지만 이 두 호출 사이에 게임서버가 죽거나 응답을 놓치면 어떻게 될까?

Coupon Platform은 인앱결제의 consume/acknowledge 패턴을 참고해 **reserve 시점에 소모를 즉시 확정**하고, 뒤이은 confirm은 지급 결과를 통보받는 용도로만 쓰도록 설계했다. 분산 트랜잭션 없이도 재시도가 항상 안전하다 — reserve를 두 번 호출해도(먹등), confirm을 영원히 안 불러도(코드는 이미 소모 완료 처리) 데이터가 어긋나지 않는다.

이 모델을 실제로 안전하게 만드는 건 결국 동시성 처리다.

- 코드 중복 소모/캠페인 오버셀 → 조건부 UPDATE + 갱락
- 관리 콘솔의 동시 수정 충돌 → `edit_count` 낙관적 동시성 제어
- 게임서버 인증 재전송 → API Key + HMAC-SHA256 서명 + Timestamp/Nonce, Secret은 유효기간을 둔 로케이션

멀티테넌트 운영 구조

회사(company) → 프로젝트(project) → 쿠폰 도메인(캠페인/코드/사용이력) 계층으로 데이터 모델을 설계했다. 코드 값 유니크 범위도 전역이 아니라 프로젝트 단위로 스코핑되어, 서로 다른 게임의 쿠폰 코드가 우연히 겹쳐도 문제가 없다.

역할은 4단계 누적 구조(SUPER_ADMIN ⊇ DEVELOPER ⊇ MANAGER ⊇ OPERATOR)로, 쿠폰 도메인 작업은 프로젝트에 실제로 배정된 역할 기준으로 스코핑된다 — 같은 회사 소속이어도 배정되지 않은 프로젝트의 쿠폰은 건드릴 수 없다.

설계 원칙

프로젝트 전반에 일관되게 적용한 아키텍처 원칙이다.

- **Stored Procedure/Function 전용** — ORM·Native SQL 직접 작성을 금지하고 모든 DB 접근을 SP/Function 으로 캡슐화해 트랜잭션 경계와 정합성을 DB 레벨에서 보장
- **물리 삭제 금지** — 모든 도메인에서 논리 삭제(`status`)만 사용, 목록 정렬도 `status DESC` 우선으로 통일
- **방어적 권한 재검증** — SP가 호출자가 전달한 `role_code`를 그대로 신뢰하지 않고, 내부에서 DB로 직접 재 확인(재사용 가능한 공용 Function으로 캡슐화)
- **낙관적 동시성 제어** — `updated_at` 같은 시간 기반 토큰 대신 전용 `edit_count` 정수 컬럼을 사용해 동 시 수정 충돌을 타이밍에 의존하지 않고 100% 감지
- **조건부 UPDATE + 락** — 코드 중복소모/캠페인 오버셀처럼 정합성이 중요한 경로는 "체크"와 "커밋" 사이 레이스가 없도록 조건부 UPDATE와 갱락으로 원자적으로 처리
- **영역별 로그 분리** — 시스템관리자(`log_audit`) / 플랫폼운영자(`log_coupon_campaign`) / 유저 (`log_coupon_use`) 3개 로그를 물리적으로 분리된 DB에 Append-Only로 기록

기술 스택

Backend

항목	스택
Runtime	Node.js 22 LTS
Framework	NestJS + TypeScript
Database	MySQL 8.4
Data Access	mysql2 (Stored Procedure/Function 전용)
인증	JWT(HS256) + user_session
S2S 인증	API Key + HMAC-SHA256 요청 서명
비밀번호	bcrypt (rounds=12)
로깅	log_audit / log_coupon_campaign / log_coupon_use + application log(log4js)

왜 SP/Function 전용으로 강제했는가 정합성이 중요한 도메인(코드 중복소모 방지, 캠페인 오버셀 방지, 낙관적 동시성 제어)일수록 애플리케이션 레이어의 트랜잭션 코드보다 DB 제약·락으로 강제하는 편이 실수할 여지가 적다. ORM이나 Native SQL 직접 작성을 금지하고 모든 쓰기를 Stored Procedure로 강제한 이유다.

Frontend

항목	스택
Framework	React 18 + TypeScript
Build	Vite
UI	Ant Design
폼	Ant Design Form
상태 관리	Zustand
HTTP	Axios

주요 기능

- **회사 / 프로젝트 관리** — 멀티테넌트 데이터 격리, 프로젝트 단위 API Key/Secret 발급
- **사용자 관리** — 가입 승인 워크플로우 / 역할 기반 권한 (SUPER_ADMIN/DEVELOPER/MANAGER/OPERATOR)
- **캠페인/쿠폰 코드 관리** — 캠페인 CRUD, RANDOM 코드 비동기 대량생성(재시도/정체 복구), FIXED 코드 동기 등록
- **쿠폰 사용(S2S)** — reserve(즉시확정) / confirm(결과통보), 미컨슈م 조회
- **감사 로그** — 시스템관리자(log_audit) / 플랫폼운영자(log_coupon_campaign) / 유저(log_coupon_use) 영역별 로그 분리

즉시 반영 vs 승인 워크플로우 기준 쿠폰 도메인 작업(캠페인 등록/수정, 코드 발급)은 요청자 역할에 따라 처리 방식이 갈린다. **MANAGER** 이상은 즉시 반영되고, **OPERATOR** 가 등록/수정하면 승인요청 상태로 전환되어 **MANAGER** 이상의 승인을 거쳐야 한다. 운영 실수(과다 발급, 잘못된 보상 설정 등)를 하위 권한자의 작업에 한해 한 번 더 걸러내기 위한 구조다.

AI 활용

이 프로젝트는 AI를 개발 보조 도구로 활용한 워크플로우를 실험하기 위해 진행되었다.

설계 결정, 트레이드오프 판단, 실서버 라이브 검증은 개발자가 직접 수행하였으며, AI(Claude Code)는 문서/코드 초안 작성과 반복적인 구현 작업의 속도를 높이는 데 활용하였다. 예를 들어 낙관적 동시성 제어를 updated_at 비교에서 edit_count 전용 컬럼으로 바꾼 결정, S2S rate limiter를 고정 윈도우에서 토큰 버킷으로 교체한 결정은 모두 개발자가 먼저 문제를 지적하고 방향을 제시한 뒤 구현했다.

도구	용도
Claude Code	문서/코드 초안 작성, 리팩터링, 실서버 스모크 테스트 보조

현재 상태

- ☒ 데이터베이스 설계 완료 — 테이블 12개, 메인 DB/로그 DB 물리 분리
- ☒ API 명세 작성 완료 — 설계 문서 21종(docs/01~21)
- ☒ 역할별 권한 설계 완료 — SUPER_ADMIN/DEVELOPER/MANAGER/OPERATOR 4단계 누적 구조
- ☒ **Backend 구현 완료**
 - 공통 인프라 — NestJS + mysql2 SP 실행기, 공통 응답 포맷, S2S HMAC 인증 가드, 역할 기반 권한 가드
 - Auth / Company / Project / User·User Role / Audit Log API 전체
 - Campaign API — CRUD·상태변경·승인/반려(edit_count 낙관적 동시성 제어)
 - Coupon Code Issuance API — RANDOM 비동기 대량생성+재시도, FIXED 동기 등록, 정체 시 수동 복구(abort)
 - Coupon Usage API(S2S) — reserve / confirm(즉시확정 모델) / 미컨슈م 조회
 - 스케일아웃(수평 확장) 대응 — graceful shutdown, 크론 배치 중복실행 방지, DB 커넥션 풀 크기 env화, 정체 감지 모니터링
 - 사용기간 만료 캠페인 자동 종료 배치
 - Swagger 문서화 — 요청/응답 스키마 전체({result,data} 응답 봉투 포함)
 - E2E 테스트 스위트 — 문서화된 42개 엔드포인트 전체를 8개 도메인/141개 테스트로 커버, 실행마다 DB 자동 리셋+재시딩

- ✔ **Frontend 구현 완료**(React 18 + TypeScript + Vite + Ant Design + Zustand + Axios)
 - 화면 목록 전체 구현 — 로그인/회원가입, 관리메뉴(회사/프로젝트/사용자/감사로그), 캠페인 목록·등록·상세(정보·코드목록·사용이력·변경이력 4개 탭), 쿠폰 사용 로그
 - 다국어(i18n) — react-i18next로 ko/en 지원
 - 실제 백엔드 연동 + 브라우저 라이브 검증 완료
- ✔ **테스트 게임서버**(test_game_server) 구축 완료 — S2S 연동을 실제 외부 서비스처럼 상시 검증하는 독립 서비스. 6개 시나리오(정상흐름/먹등재시도/동시성레이스/소진쿠폰재시도/보상지급중단 후 재처리/에러케이스)를 가중치 기반으로 반복 실행하며, 재사용 가능한 참고 SDK(CouponS2sClient)를 함께 제공
- ✔ **입점 게임사용 이용/연동 가이드 작성 완료** — 이 문서의 Part 2

문서 목록

파일	설명
01_COUPON_PLATFORM_GUIDE.md	입점 게임사 담당자용 이용/연동 가이드(콘솔 사용법 + S2S 연동)
02_TECH_STACK.md	기술 스택, 환경변수
03_DEV_SETUP.md	로컬 개발 환경 설정
04_DEV_CONVENTIONS.md	로깅 원칙, 코드 모듈화 기준, SP 네이밍, 동시성 처리 컨벤션
05_ERD.md	전체 테이블 ERD, 비정규화 FK, 상태코드 요약
06_DATABASE_SCHEMA.md	테이블별 특징/상태/특수규칙
07_COUPON_ISSUANCE_SCENARIO.md	캠페인/코드 발급 흐름, 비동기 생성, 재시도 처리
08_COUPON_USAGE_SCENARIO.md	쿠폰 사용 흐름, 동시성 처리
09_AUTH_SECURITY.md	사용자 인증, S2S 인증 정책
10_API_COMMON.md	응답포맷/에러코드/페이지네이션
11_AUTH_API.md	회원가입/로그인/로그아웃 등
12_COMPANY_API.md	회사 CRUD
13_PROJECT_API.md	프로젝트 CRUD, Secret 발급/재발급
14_USER_API.md	사용자 승인/반려/권한 배정
15_LOG_AUDIT_API.md	감사로그 조회
16_MENU_PERMISSION.md	역할별 메뉴 접근 권한
17_SCREEN_LIST.md	화면 목록 및 연관 API
18_LAYOUT.md	레이아웃, 라우트, 공통 컴포넌트
19_CAMPAIGN_API.md	캠페인 CRUD, 상태변경, 승인/반려, 코드 발급(RANDOM 비동기/FIXED 동기)
20_COUPON_USAGE_API.md	쿠폰 사용 reserve/confirm, 미컨sum 조회
21_TEST_GAME_SERVER.md	S2S 연동 검증용 독립 테스트 클라이언트(test_game_server) 설계

프로젝트 구조

```
coupon_platform/
├── backend/          # Backend (NestJS)
├── frontend/         # Frontend (React)
├── database/
│   ├── tables/       # 메인 DB DDL(.sql, all_tables.sql 포함)
│   ├── procedures/   # 메인 DB Stored Procedure
│   └── functions/    # 메인 DB Function
├── database_log/
│   ├── tables/       # 로그 DB DDL(메인 DB와 물리 분리)
│   └── procedures/   # 로그 DB 전용 Stored Procedure
├── test_game_server/ # S2S 연동 검증용 독립 테스트 클라이언트(backend/frontend와 완전 분리)
└── docs/             # 설계 문서
```

Part 2. 사용 가이드

아래는 입점(연동) 게임사 담당자를 위해 작성한 실사용 문서입니다. Coupon Platform이 무엇인지, 어떻게 가입하고 로그인하는지, 역할별로 화면에서 무엇을 할 수 있는지, 그리고 게임서버를 Coupon Platform에 연동하려면 백엔드 개발자가 무엇을 해야 하는지를 다룹니다.

목차

- [1. Coupon Platform이란?](#)
- [2. 시작하기](#)
 - [2.1 계정 발급 절차](#)
 - [2.2 계정 상태](#)
- [3. 화면 기본 구성](#)
- [4. 역할과 할 수 있는 일](#)
- [5. 관리 업무 \(SUPER ADMIN·DEVELOPER\)](#)
 - [5.1 회사·프로젝트](#)
 - [5.2 사용자·권한](#)
 - [5.3 게임서버 연동 정보\(API Key/Secret\) 관리](#)
- [6. 쿠폰 운영 업무 \(전체 역할\)](#)
 - [6.1 캠페인 등록과 승인 워크플로우](#)
 - [6.2 쿠폰 코드 발급](#)
 - [6.3 사용 이력 / 변경 이력 / 쿠폰 사용 로그 조회](#)
- [7. 감사 로그 \(SUPER ADMIN·DEVELOPER\)](#)
- [8. 게임서버 개발자를 위한 연동 안내](#)
 - [8.1 공통 API 계약](#)
 - [8.2 S2S 인증 서명 만들기](#)
 - [8.3 쿠폰 사용 API \(reserve / confirm / 미션습 조회\)](#)
 - [8.4 재시도와 먹등성 — 반드시 알아야 할 함정](#)
 - [8.5 연동 절차 요약 \(체크리스트\)](#)
- [9. 자주 묻는 질문](#)
- [10. 문의](#)
- [11. 부록 A. CouponS2sClient 참고 SDK 전체 소스](#)

1. Coupon Platform이란?

Coupon Platform은 여러 게임이 하나의 플랫폼을 공유해 쓰는 **멀티테넌트 쿠폰 발급/사용 서버**입니다.

신규가입 보상, 프로모션 코드처럼 게임마다 반복해서 필요한 "쿠폰 코드를 만들고, 유저가 입력한 코드를 검증해서 보상을 지급하는" 기능을, 게임마다 각자 구현하지 않고 이 플랫폼 하나로 통합해서 처리합니다.

운영팀이 관리 콘솔에서 캠페인/쿠폰코드를 등록 → 게임서버가 유저의 코드 입력을 이 서버에 검증 요청 → 유저에게 보상 지급

즉, 여러분의 게임서버는 유저가 쿠폰 코드를 입력하는 시점에 이 서버를 호출해 "이 코드가 유효하고 아직 안 쓰였는지"만 확인받으면 되고, 코드 중복 소모 방지·발급 수량 초과 방지·서로 다른 게임 간 데이터 격리는 플랫폼이 구조적으로 보장합니다.

회사(company) → 프로젝트(project, 게임 타이틀 단위) → 캠페인/쿠폰코드 계층으로 데이터가 격리되어 있어, 여러 게임 타이틀을 하나의 계정 체계로 통합 운영할 수도 있습니다.

2. 시작하기

2.1 계정 발급 절차

Coupon Platform은 사내·협력사 인원만 사용하는 폐쇄형 도구라, 아무나 회원가입만으로 즉시 사용할 수는 없습니다.

1. Coupon Platform 운영 담당자(SUPER_ADMIN)에게 **회사코드**(및 필요 시 프로젝트코드)를 요청합니다.
2. /signup 화면에서 회원가입을 신청합니다. 이때 회사코드/프로젝트코드는 드롭다운이 아니라 **직접 입력**합니다 — 로그인 전 화면이라 전체 회사·프로젝트 목록을 공개하지 않기 위한 설계입니다. 프로젝트코드는 선택 입력입니다.
3. 가입 신청 직후 계정 상태는 **가입승인대기**입니다. SUPER_ADMIN이 승인해야 로그인할 수 있습니다.
4. 승인이 완료되면 등록된 로그인 ID/비밀번호로 로그인합니다.

아래는 로그인/회원가입 화면 예시입니다.

로그인 화면 (그림 1)	회원가입 화면 (그림 2)

가입 승인만으로는 업무 화면이 비어 있을 수 있습니다. 가입 승인은 "이 사람이 로그인할 수 있다"만 결정할 뿐, 쿠폰 도메인(캠페인/코드) 업무 권한은 별도로 SUPER_ADMIN이 프로젝트 단위로 배정해야 합니다(5.2 참고). 아직 배정받지 못한 상태에서는 회사만 있고 연결된 프로젝트가 없어, 캠페인 목록 등 프로젝트 단위 화면에 아무것도 표시되지 않습니다.

2.2 계정 상태

값	의미	로그인 가능 여부
0	가입승인대기	불가
1	정상(가입승인)	가능
2	가입반려	불가
3	사용중지	불가

가입이 반려되었더라도 관리자가 상태를 다시 "가입승인대기" 등으로 되돌려주면 같은 아이디로 재승인 절차를 밟을 수 있습니다.

3. 화면 기본 구성

로그인하면 아래와 같은 공통 레이아웃이 나옵니다.

Coupon Platform · 회사 ▼ · 프로젝트 ▼ · 서버시각 · 관리 · 이름 ▼	
Sidebar (관리메뉴 / 캠페인 / 쿠폰 사용 로그)	Content

- **헤더:** 왼쪽에 회사/프로젝트 선택 콤보박스가 있습니다. SUPER_ADMIN만 "전체 회사"/"전체 프로젝트"를 선택할 수 있고, 그 외 역할은 본인 소속 회사로 고정됩니다. 캠페인 목록 등 대부분의 화면은 여기서 선택한 프로젝트를 기준으로 동작하므로, 원하는 프로젝트가 먼저 선택되어 있는지 항상 확인해야 합니다 — "전체 프로젝트" 상태로는 캠페인 목록·쿠폰 사용 로그 화면에 진입할 수 없고, 프로젝트를 먼저 고르라는 안내가 표시됩니다.
- **사이드바:** "관리" 메뉴(회사/프로젝트/사용자/감사로그 — 역할에 따라 일부만 보임)와, 실제 쿠폰 운영 업무를 위한 일반 메뉴(캠페인 목록/쿠폰 사용 로그)로 나뉩니다.
- **내 계정:** 헤더 우측의 이름을 클릭하면 내 정보 조회, 비밀번호 변경, 로그아웃 메뉴가 나옵니다. 비밀번호를 변경하면 보안을 위해 그 즉시 모든 세션이 종료되므로, 변경 직후에는 다시 로그인해야 합니다.
- 관리 화면(등록·상세·수정)에 들어가 있는 동안에는 헤더의 회사/프로젝트 선택이 잠깁니다 — 작업 도중 헤더 선택이 바뀌면 보고 있는 내용과 어긋나 혼란을 줄 수 있기 때문입니다. 목록 화면에서만 자유롭게 바꿀 수 있습니다.

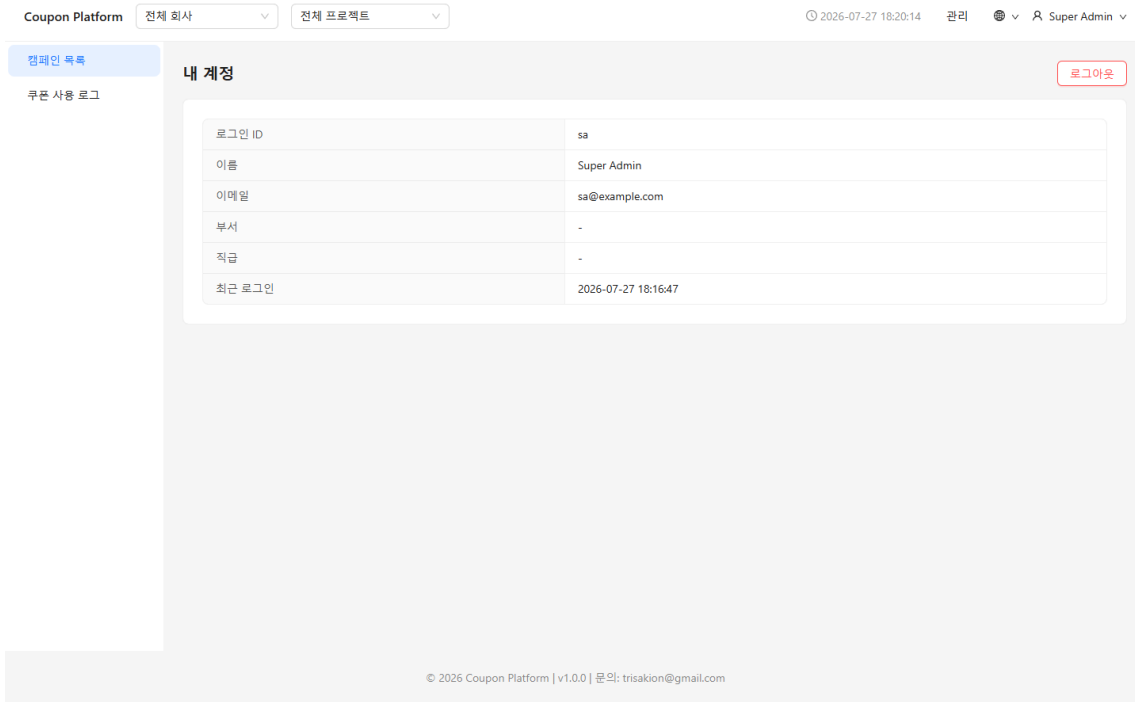


그림 2-1. 내 계정 화면(로그인 ID, 이름, 최근 로그인 등 조회 + 로그아웃)

4. 역할과 할 수 있는 일

Coupon Platform은 4단계 역할을 사용합니다. 숫자가 작을수록 고권한이며, 상위 역할이 하위 역할의 권한을 모두 포함하는 누적 구조입니다(SUPER_ADMIN ≥ DEVELOPER ≥ MANAGER ≥ OPERATOR). 단, 이 누적은 회사/프로젝트 관리 메뉴에는 역방향으로 적용되지 않습니다 — DEVELOPER 전용 권한(프로젝트 조회/Secret 재발급)이 MANAGER/OPERATOR에게 내려가지는 않습니다.

한 사용자가 프로젝트마다 다른 역할을 가질 수 있습니다(예: A 게임에서는 매니저, B 게임에서는 운영자).

역할	코드	한 줄 설명
SUPER_ADMIN	10	전체 회사·프로젝트 관리자. 계정 승인, 회사/프로젝트 생성, 권한 배정 등 플랫폼 자체를 관리
DEVELOPER	20	프로젝트 기술 담당자. 자신이 배정된 프로젝트의 조회 + API Key/Secret 재발급 담당
MANAGER	30	쿠폰 도메인(캠페인/코드) 즉시 컨트롤 — 승인 절차 없이 바로 반영, 하위 역할의 요청 승인도 담당
OPERATOR	40	쿠폰 도메인 실무 등록 담당 — 등록/수정한 내용은 승인요청 상태로 전환되어 상위 역할의 승인이 필요

역할별로 어떤 기능에 접근할 수 있는지 요약하면 다음과 같습니다.

기능	SUPER_ADMIN	DEVELOPER	MANAGER	OPERATOR
----	-------------	-----------	---------	----------

회사 등록/수정, 목록·상세 조회	○ (전체)	-	-	-
프로젝트 등록/수정	○	-	-	-
프로젝트 목록·상세 조회	○ (전체)	○ (역할보유 프로젝트만)	-	-
API Secret 재발급	○	○ (역할보유 프로젝트만)	-	-
사용자 가입 승인/반려, 권한 배정	○	-	-	-
사용자 목록·상세 조회	○ (전체)	○ (본인 회사만)	-	-
감사 로그 조회	○ (전체)	○ (자사만, 일부 항목 추가 제한)	-	-
캠페인/코드 목록·상세 조회	○	○	○	○
캠페인 등록·수정, 코드 발급/재시도	○ (즉시 반영)	○ (즉시 반영)	○ (즉시 반영)	○ (승인요청 상태로 전환)
캠페인 승인/반려	○	○	○	-
사용 이력 / 변경 이력 / 쿠폰 사용 로그 조회	○	○	○	○

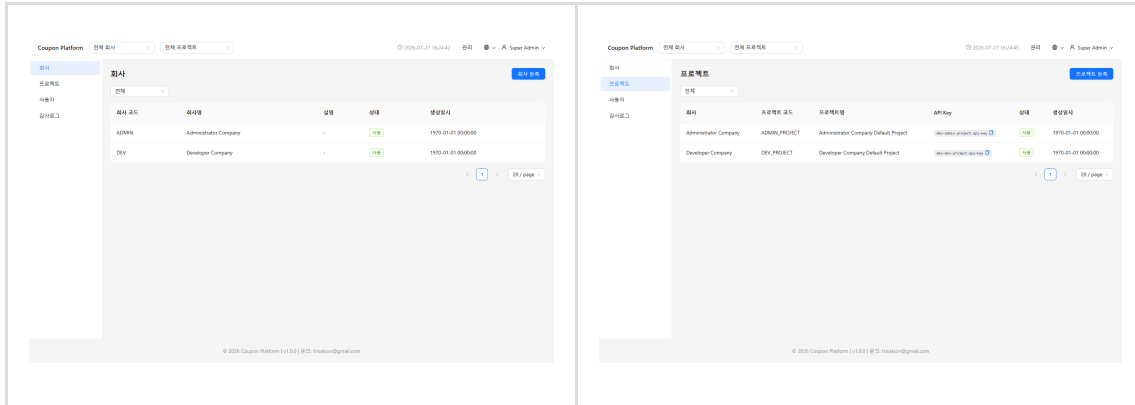
이 문서에서는 이 표를 바탕으로 실무에서 자주 쓰는 흐름 위주로 안내합니다.

5. 관리 업무 (SUPER_ADMIN·DEVELOPER)

5.1 회사·프로젝트

SUPER_ADMIN이 회사와 프로젝트(게임 타이틀)를 등록합니다. 프로젝트는 "회사 소속의 게임 하나"에 대응하며, 게임서버 연동에 필요한 API Key / API Secret 도 프로젝트 단위로 발급·관리됩니다 — 자세한 발급 절차는 5.3에서 이어서 다룹니다.

회사 목록 (그림 3)	프로젝트 목록 (그림 4)
--------------	----------------

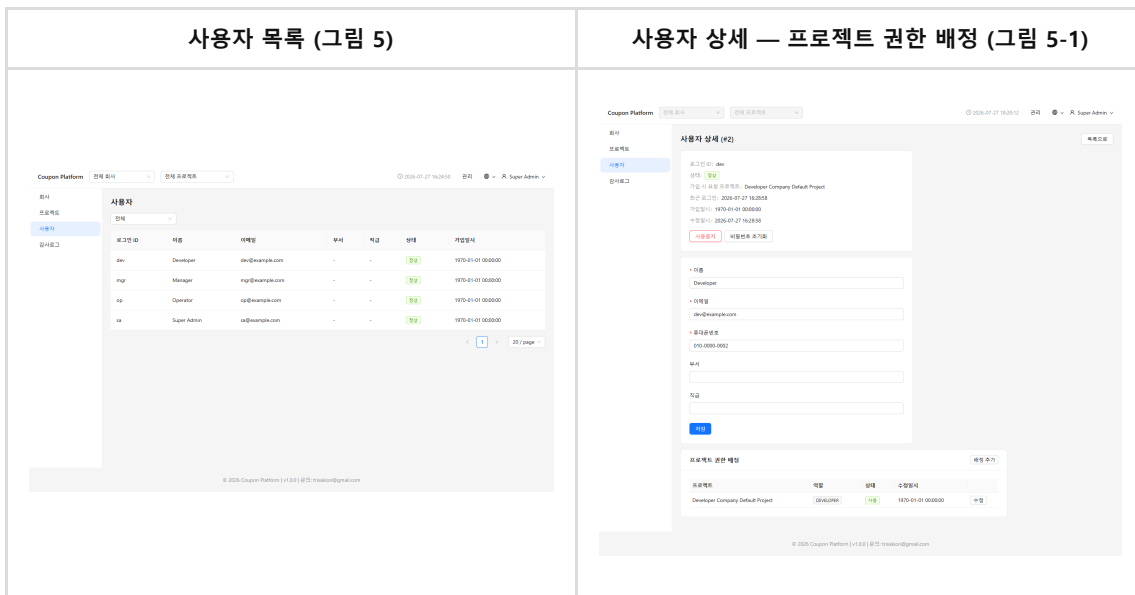


프로젝트를 새로 생성하면 `api_key` / `api_secret` 이 서버에 의해 즉시 자동 발급됩니다(회사와 달리 별도 발급 버튼이 없습니다) — 이 값들 없이는 프로젝트가 만들어지지 않는 필수 항목이기 때문입니다.

5.2 사용자·권한

SUPER_ADMIN은 가입 신청을 승인/반려하고, 사용자별로 프로젝트마다 다른 역할 (DEVELOPER/MANAGER/OPERATOR)을 부여합니다. DEVELOPER는 본인 소속 회사의 사용자 목록을 조회할 수 있습니다(수정·승인·권한배정 권한은 없습니다).

비밀번호를 잊었거나 계정에 이상이 있을 때는 SUPER_ADMIN이 강제로 비밀번호를 초기화할 수 있습니다.



사용자 상세 화면 하단의 "프로젝트 권한 배정" 표에서 이 사용자가 어떤 프로젝트에 어떤 역할 (DEVELOPER/MANAGER/OPERATOR)로 배정되어 있는지 확인하고, "배정 추가" 버튼으로 새 프로젝트 권한을 부여합니다. 2.1에서 언급한 "가입 승인만으로는 화면이 비어 있을 수 있다"는 안내가 바로 이 배정 여부를 가리킵니다 — 이 표에 아무 프로젝트도 없으면 캠페인 목록 등 프로젝트 단위 화면에 접근할 수 없습니다.

5.3 게임서버 연동 정보(API Key/Secret) 관리

프로젝트 상세 화면에서 게임서버 연동에 필요한 두 값을 확인·관리합니다.

- **API Key**(X-API-Key): 프로젝트를 식별하는 공개 식별자. 목록/상세 화면에 항상 그대로 노출됩니다.

- **API Secret:** 게임서버가 요청을 서명할 때 쓰는 값. 평문은 발급/재발급 응답에 딱 한 번만 노출되며, 이후에는 어떤 조회 화면에서도 다시 확인할 수 없습니다(발급 여부만 확인 가능합니다).

두 항목 모두 SUPER_ADMIN뿐 아니라 그 프로젝트에 실제로 DEVELOPER 이상 권한을 가진 담당자가 직접 관리할 수 있습니다 — 실제로 게임서버를 만드는 쪽이 매번 SUPER_ADMIN을 거치지 않고 스스로 Secret을 재발급할 수 있어야 한다는 판단입니다. 같은 회사 소속이거나 다른 프로젝트에서 DEVELOPER 이상이더라도, 해당 프로젝트에 배정되지 않았다면 이 값을 볼 수 없습니다.

Coupon Platform 전체 회사 전체 프로젝트 2026-07-27 16:24:46 관리 Super Admin

회사 프로젝트 사용자 감사로그

프로젝트 상세 (#2) 목록으로

회사: Developer Company
 프로젝트 코드: DEV_PROJECT
 API Key: dev-dev-project-api-key
 Secret 재발급 일시: 재발급 이력 없음
 API Secret 재발급

* 프로젝트명
 Developer Company Default Project

설명

상태
 사용
 저장

생성 일시: 1970-01-01 00:00:00
 수정 일시: 2026-07-27 14:06:19

© 2026 Coupon Platform | v1.0.0 | 문의: trisakion@gmail.com

그림 6. 프로젝트 상세 — API Key 및 재발급 버튼

API Secret 재발급 방법

1. 프로젝트 상세 화면에서 "API Secret 재발급" 버튼을 누릅니다.
2. 재발급 확인창에는 유예기간(기본값은 관리자에게 문의, 화면에 실제 값이 표시됩니다) 안내가 함께 뜹니다 — 재발급 이후에도 이 기간 동안은 이전 Secret으로 서명한 요청도 함께 유효합니다(아래 참고).

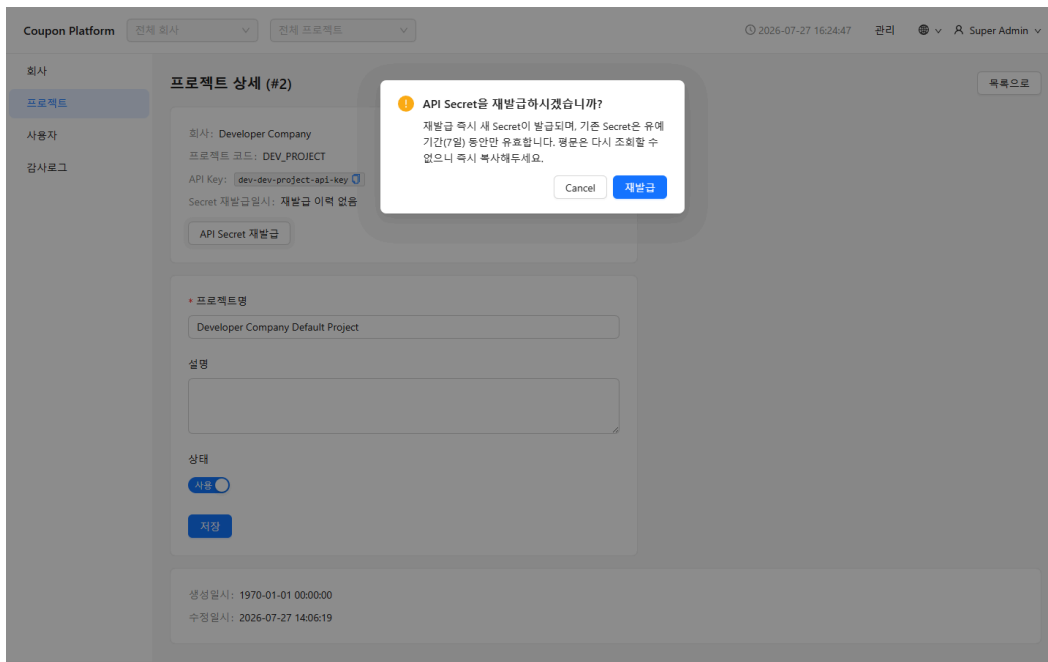


그림 7. API Secret 재발급 확인창

3. 확인을 누르면 새 Secret이 생성되고, 이번 한 번만 평문으로 화면에 표시됩니다.

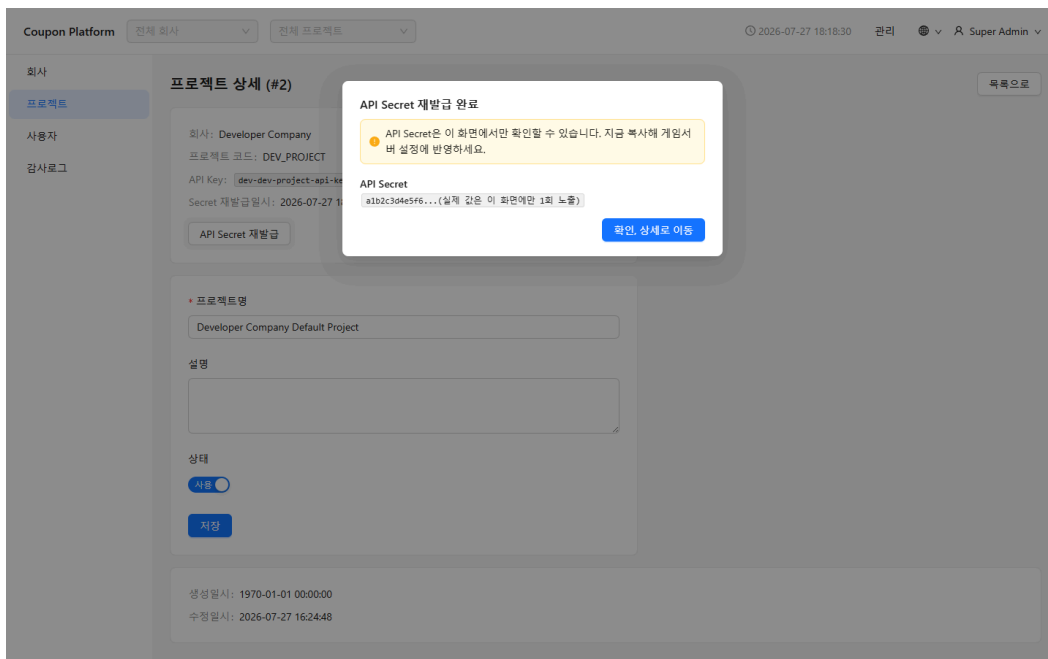


그림 8. 재발급된 API Secret 1회 노출 화면(실제 값은 예시를 위해 가림 처리했습니다)

4. 이 값을 복사해 여러분의 게임서버 설정(환경변수 등)에 그대로 붙여넣습니다. 창을 닫으면 이후로는 어디에서도 평문을 다시 확인할 수 없습니다.

주의: 재발급은 역등하지 않습니다 — 재발급 버튼을 누를 때마다 이전 Secret이 유효기간용으로 한 칸 밀려나므로, 같은 요청을 실수로 두 번 연속 보내면 두 번째는 "다른 관리자가 먼저 수정함"으로 거부됩니다(화면을 새로

고침한 뒤 다시 시도하면 됩니다). 이는 버그가 아니라, 짧은 시간에 두 번 재발급되어 아직 유효했던 이전 Secret이 흔적도 없이 사라지는 사고를 막기 위한 안전장치입니다.

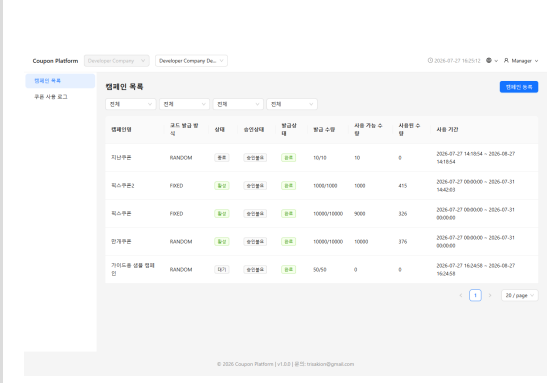
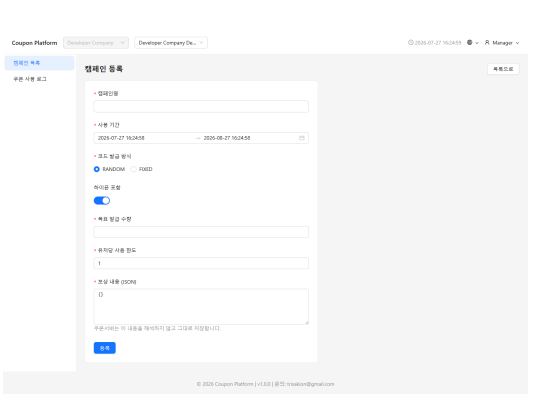
Secret 로테이션(교체)은 유예기간(Grace Period) 방식으로 동작합니다 — 재발급 직후부터 유예기간이 끝날 때까지는 새 Secret과 이전 Secret 둘 다로 서명한 요청이 통과됩니다. 게임서버 배포가 여러 대의 서버에 순차적으로 이루어지는 경우에도, 일부 서버가 아직 이전 Secret으로 서명하고 있는 동안 인증이 끊기지 않도록 하기 위한 설계입니다. 유예기간이 지나면 이전 Secret은 자동으로 폐기되며, 그 전까지는 반드시 게임서버 설정을 새 Secret으로 교체해야 합니다.

게임서버 쪽에서 이 값을 실제로 어떻게 쓰는지 8장을 참고합니다.

6. 쿠폰 운영 업무 (전체 역할)

6.1 캠페인 등록과 승인 워크플로우

"캠페인"은 하나의 쿠폰 이벤트 단위입니다(사용기간, 코드 발급방식, 유저당 사용한도, 보상 내용을 포함). 사이드바의 "캠페인" 메뉴에서 목록/등록/상세를 다룹니다.

캠페인 목록 (그림 9)	캠페인 등록 (그림 10)
	

등록 시 입력하는 주요 항목:

항목	설명
캠페인명	자유 입력
사용기간	시작/종료 일시. 종료 일시가 시작보다 빨라서는 안 됨
코드 발급 방식	RANDOM(무작위 코드 대량생성) 또는 FIXED(관리자가 지정한 고정 코드 1개)
수량	RANDOM은 생성할 코드 개수, FIXED는 그 코드 1개가 지원할 총 사용가능 횟수(코드 개수가 아님 — 6.2 참고)
유저당 사용한도	같은 유저가 같은 캠페인을 몇 번 사용할 수 있는지
보상 내용	JSON 형태로 자유롭게 입력. 플랫폼은 이 내용을 해석하지 않고 그대로 저장했다가 게임서버에 그대로 돌려줍니다 — 실제 지급 처리는 게임서버가 담당합니다

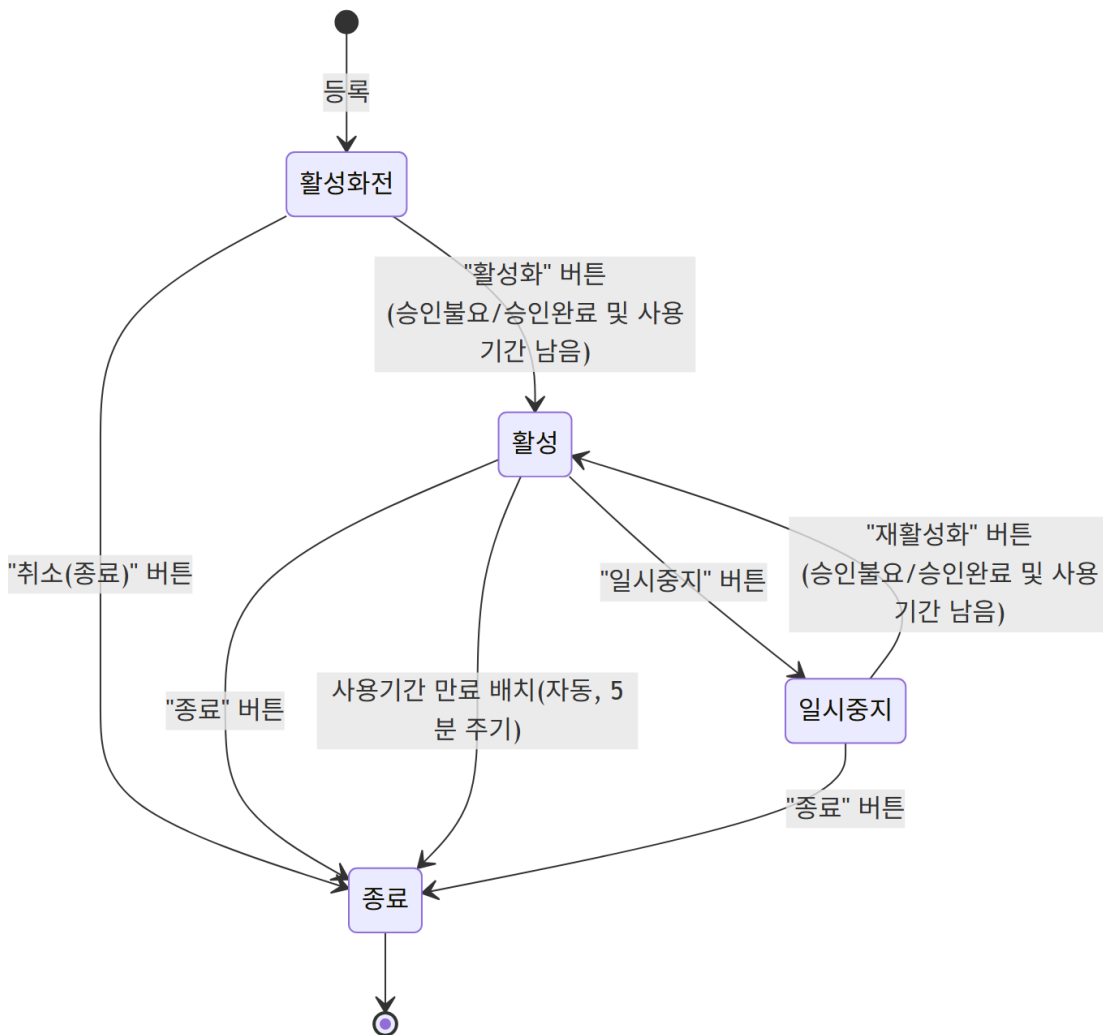
등록/수정한 내용이 즉시 반영되는지, 승인이 필요한지는 요청자의 역할에 따라 갑니다.

- MANAGER 이상이 등록/수정하면 **즉시 반영**됩니다(승인불요).
- OPERATOR가 등록/수정하면 **승인요청 상태**로 전환됩니다 — SUPER_ADMIN·DEVELOPER·MANAGER 중 누군가 캠페인 상세 화면에서 승인/반려해야 실제로 활성화할 수 있습니다. 반려된 캠페인은 OPERATOR가 다시 수정해서 재상신할 수 있습니다(수정 즉시 다시 승인요청 상태가 됩니다).
- 이미 승인된 캠페인이 **활성** 상태로 서비스 중인 도중 OPERATOR가 수정을 시도하면, 저장 즉시 캠페인이 다시 승인대기 상태로 전환되면서 **자동으로 일시중지**됩니다 — 미승인 내용이 유저에게 그대로 노출되는 사고를 막기 위한 안전장치입니다. 화면에는 저장 전에 이 사실을 알리는 확인창이 먼저 뜹니다.

캠페인 상태(활성화 전 / 활성화 / 일시중지 / 종료)는 별도 버튼으로 전환합니다. 종료된 캠페인은 어떤 수정도 다시 허용되지 않는 최종 상태입니다 — 되돌릴 수 없으니 새 캠페인을 만들어야 합니다. 또한 **사용기간이 지난 캠페인은 배치가 자동으로 종료** 처리합니다(5분 주기 점검, 별도 조작 불필요).

위 규칙을 화면 버튼 기준의 "상태 전이"와, 그와 별개 축인 "승인 워크플로우" 두 다이어그램(+ 표)으로 정리하면 다음과 같습니다.

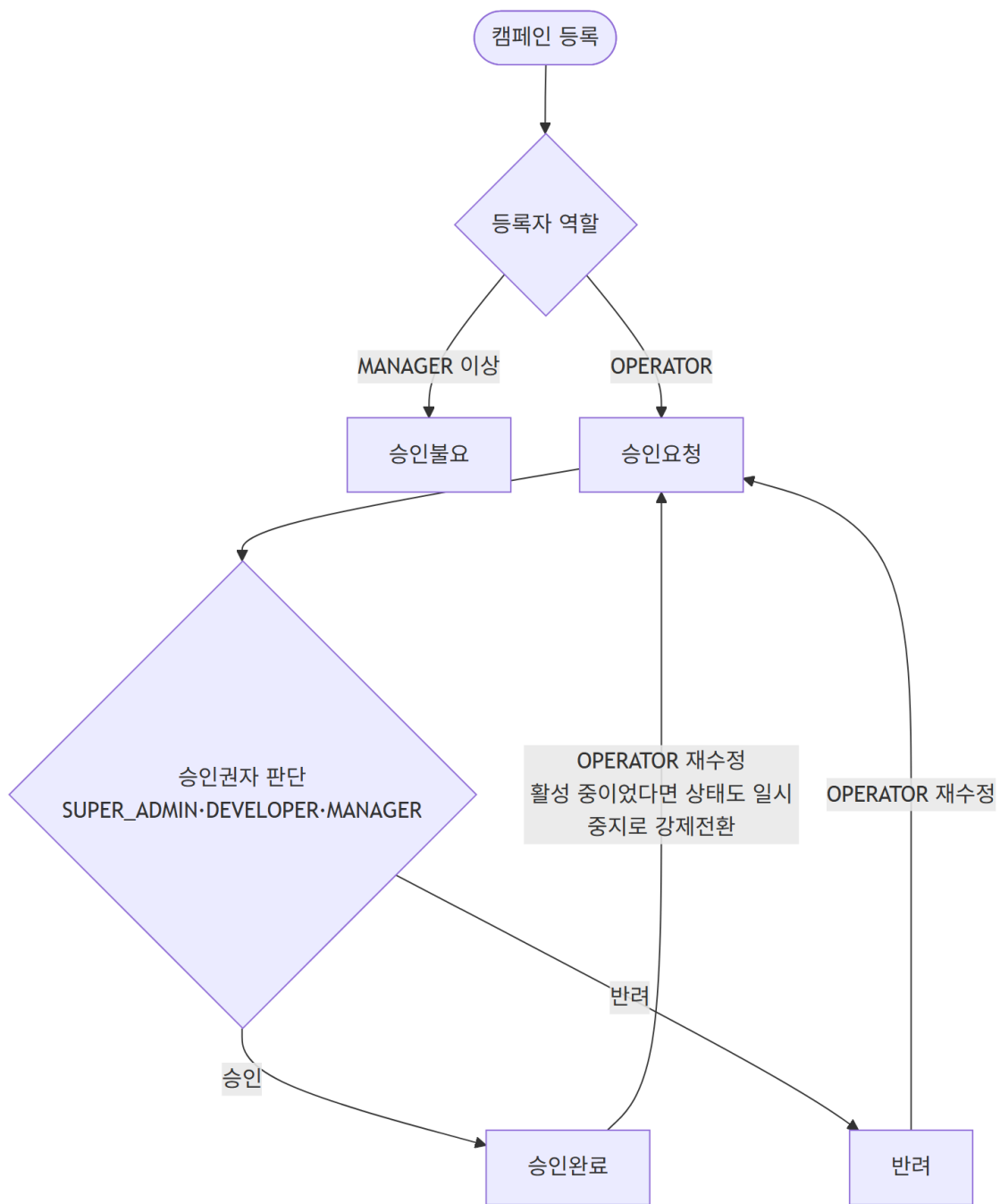
상태 전이(활성화 전 / 활성화 / 일시중지 / 종료)



현재 상태	버튼 / 트리거	다음 상태	조건
활성화 전	"활성화"	활성	승인상태가 승인불요/승인완료여야 하고, 사용기간(campaign_end)이 아직 남아있어야 함
활성화 전	"취소(종료)"	종료(최종)	승인상태·사용기간과 무관하게 즉시 가능 — 활성화해볼 필요조차 없는 캠페인을 접을 때 씁니다
활성	"일시중지"	일시중지	-
활성	"종료"	종료(최종)	-
활성	(자동) 사용기간 만료 배치	종료(최종)	5분 주기 점검, 별도 조작 불필요(승인불요/승인완료 상태였던 캠페인만 대상)
일시중지	"재활성화"	활성	승인상태가 승인불요/승인완료여야 하고, 사용기간이 아직 남아있어야 함
일시중지	"종료"	종료(최종)	-

종료는 되돌릴 수 없는 최종 상태입니다 — 다시 쓰려면 새 캠페인을 만들어야 합니다. "취소(종료)"와 "종료"는 버튼 라벨이 다른 별개의 버튼이니 헷갈리지 않도록 주의하세요(전자는 활성화 전 상태에서만, 후자는 활성/일시중지 상태에서만 보입니다).

승인 워크플로우(승인불요 / 승인요청 / 승인완료 / 반려 , 위 상태 전이와는 다른 축입니다)



현재 상태	트리거	다음 상태	부수효과
(등록)	MANAGER 이상이 등록	승인불요	승인 절차 없이 바로 시작

6.2 쿠폰 코드 발급

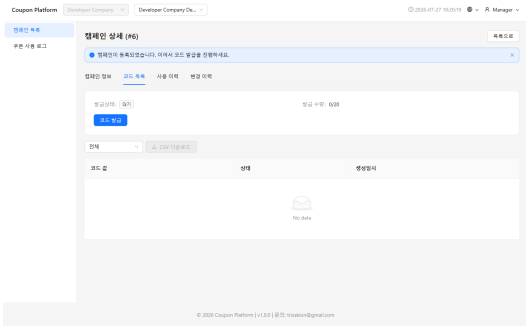
캠페인을 등록한 뒤, 상세 화면의 "코드 목록" 탭에서 실제 쿠폰 코드를 발급합니다.

- **RANDOM**: "코드 발급" 버튼을 누르면 즉시 접수되고, 백그라운드에서 지정한 수량만큼 무작위 코드를 순차 생성합니다(대량 생성이라 시간이 걸릴 수 있습니다). 발급 현황(진행 중/완료/실패)이 화면에 표시되며, 실패 시 "재시도" 버튼으로 이어서 생성할 수 있습니다. 아주 드물게 발급 작업이 멈춘 것으로 보이면 "발급 중단" 버튼으로 강제 정리 후 재시도할 수 있습니다.
- **FIXED**: "코드 등록" 버튼으로 코드 값을 직접 입력해 등록합니다(예: SUMMER2024). 캠페인당 코드는 항상 1건뿐이지만, 등록 시 지정한 수량만큼 서로 다른 여러 유저가 각자 독립적으로 그 코드를 사용할 수 있습니다 — "코드 1건 = 발급된 쿠폰 1장"이 아니라 "코드 1건 = 그 코드가 지원하는 총 사용 횟수"라는 뜻입니다.

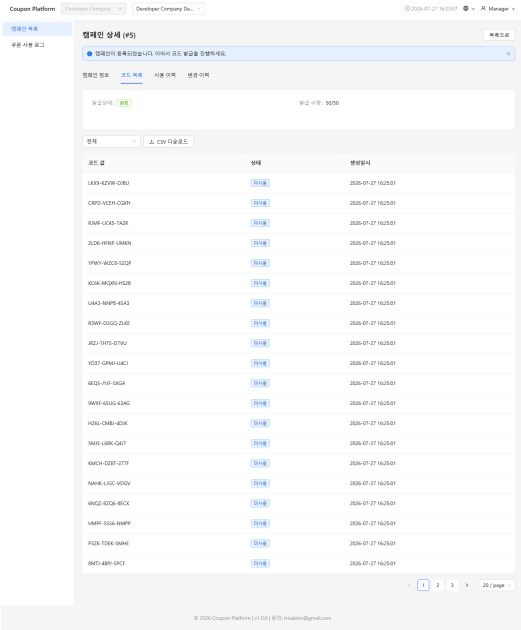
코드 목록 탭에는 "CSV 다운로드" 버튼도 있어, 발급된 코드 전체를 파일로 내려받아 별도로 관리하거나 유저에게 배포하는 용도로 쓸 수 있습니다.

코드 발급 후 상세 화면 "캠페인 정보" 탭에서 usable_qty (사용가능수량)를 열어야 실제로 게임서버가 그 코드를 사용할 수 있습니다 — 코드가 만들어진 상태와 실제 유저에게 오픈된 상태는 별개 값으로 관리됩니다(발급된 수량 이하로만 열 수 있습니다).

발급 전 — 대기 (그림 12)



발급 완료 후 (그림 13)



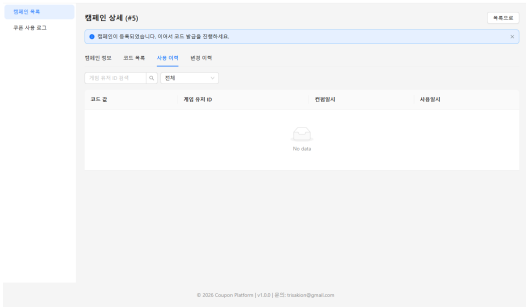
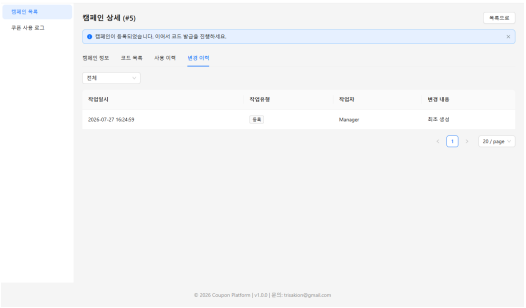
발급 전에는 "발급상태: 대기"와 "코드 발급" 버튼만 보이고 코드 목록은 비어 있습니다. "코드 발급"을 누르면 백그라운드 생성이 시작되고, 완료되면 오른쪽 그림처럼 목표 수량만큼의 코드가 "미사용" 상태로 채워집니다.

6.3 사용 이력 / 변경 이력 / 쿠폰 사용 로그 조회

캠페인 상세 화면에는 조회 전용 탭 2개가 더 있습니다.

- **사용 이력**: 이 캠페인의 쿠폰이 실제로 소모된 기록입니다. 유저 ID, 지급결과 통보(confirm) 여부로 필터링할 수 있습니다.

- **변경 이력:** 이 캠페인의 등록/수정/상태변경/승인/반려 이력입니다. 각 시점의 캠페인 전체 스냅샷이 남아, 언제 무엇이 바뀌었는지 시간순으로 추적할 수 있습니다.

사용 이력 탭 (그림 14)	변경 이력 탭 (그림 15)
	

사이드바의 별도 화면 "쿠폰 사용 로그"는 특정 캠페인에 한정되지 않고 **프로젝트 전체**의 사용 시도를 보여줍니다 — 성공한 사용뿐 아니라 실패한 시도(존재하지 않는 코드, 이미 소모된 코드, 한도 초과 등)까지 전부 남아 있어, 코드 브루트포스 시도나 이상 징후를 확인하거나 게임서버 개발자의 연동 문의에 대응할 때 유용합니다.

캠페인 목록

쿠폰 사용 로그

쿠폰 사용 로그

캠페인 ID ...

게임 유저 ID 검색

코드 값 검색

전체

전체

Start date → End date

요청일시	작업유형	캠페인	코드 값	게임 유저 ID	결과	호출자 IP
2026-07-27 15:15:30	예약(RESERVE)	픽스쿠폰	FIX_TEST	test_game_server_player_22	성공	::1
2026-07-27 15:15:30	예약(RESERVE)	픽스쿠폰 2	FIX_TEST2	test_game_server_player_36	사용자한도초과	::1
2026-07-27 15:15:30	예약(RESERVE)	픽스쿠폰 2	FIX_TEST2	race_fixed_1785132930193	성공	::1
2026-07-27 15:15:30	예약(RESERVE)	만개쿠폰	UZF8-5DF4-CFD5	test_game_server_player_9	사용자한도초과	::1
2026-07-27 15:15:30	예약(RESERVE)	만개쿠폰	7XU5-WUZ9-YCHF	test_game_server_player_26	사용자한도초과	::1
2026-07-27 15:15:29	예약(RESERVE)	픽스쿠폰	FIX_TEST	test_game_server_player_28	성공	::1
2026-07-27 15:15:29	예약(RESERVE)	픽스쿠폰	FIX_TEST	test_game_server_player_48	성공	::1
2026-07-27 15:15:29	예약(RESERVE)	만개쿠폰	ZXRM-XN4V-9UQM	test_game_server_player_8	사용자한도초과	::1
2026-07-27 15:15:29	예약(RESERVE)	픽스쿠폰	FIX_TEST	test_game_server_player_41	성공	::1
2026-07-27 15:15:29	예약(RESERVE)	픽스쿠폰	FIX_TEST	race_fixed_1785132929538	성공	::1
2026-07-27 15:15:29	예약(RESERVE)	픽스쿠폰 2	FIX_TEST2	test_game_server_player_10	사용자한도초과	::1
2026-07-27 15:15:29	예약(RESERVE)	-	NO_SUCH_CODE_72d901c5-53ea-4f06-88a4-7656c4228ca0	test_game_server_player_48	코드없음	::1
2026-07-27 15:15:29	예약(RESERVE)	만개쿠폰	VMGT-K4CC-7547	test_game_server_player_42	사용자한도초과	::1
2026-07-27 15:15:29	예약(RESERVE)	만개쿠폰	EHGN-9UEA-NHD8	test_game_server_player_37	사용자한도초과	::1
2026-07-27 15:15:29	예약(RESERVE)	픽스쿠폰 2	FIX_TEST2	test_game_server_player_21	사용자한도초과	::1
2026-07-27 15:15:28	예약(RESERVE)	픽스쿠폰	FIX_TEST	test_game_server_player_17	성공	::1
2026-07-27 15:15:28	예약(RESERVE)	픽스쿠폰 2	FIX_TEST2	race_fixed_1785132928785	성공	::1
2026-07-27 15:15:28	예약(RESERVE)	픽스쿠폰	FIX_TEST	test_game_server_player_44	성공	::1
2026-07-27 15:15:28	예약(RESERVE)	만개쿠폰	NRSW-LB6Q-MYQK	test_game_server_player_45	사용자한도초과	::1
2026-07-27 15:15:28	예약(RESERVE)	픽스쿠폰 2	FIX_TEST2	test_game_server_player_23	사용자한도초과	::1

그림 16. 쿠폰 사용 로그(프로젝트 전체, 실패 시도 포함)

7. 감사 로그 (SUPER_ADMIN·DEVELOPER)

회사/프로젝트/사용자/권한 배정 등 관리 데이터의 모든 변경 이력이 수정·삭제 없이 계속 쌓이기만 하는 방식 (Append-Only)으로 기록됩니다. 변경 전/후 값을 함께 확인할 수 있어, "언제 누가 무엇을 바꿨는지" 추적이 필요할 때 사용합니다.

로그 자체가 수정·삭제되지 않으므로, 운영 사고가 발생했을 때 "그 값을 누가 언제 바꿨는지"를 사후에도 그대로 신뢰할 수 있는 근거 자료로 쓸 수 있습니다.

DEVELOPER는 본인 소속 회사의 로그만 조회할 수 있고, 그중 프로젝트/권한배정 관련 로그는 본인이 실제로 담당하는(DEVELOPER 이상 권한을 가진) 프로젝트로 한 번 더 좁혀집니다 — 담당하지도 않는 프로젝트의 변경 이력까지 보이지 않도록 하기 위함입니다.

캠페인/쿠폰코드 자체의 변경 이력은 이 화면이 아니라 6.3의 캠페인 상세 "변경 이력" 탭에서 확인합니다(별도 영역으로 분리 관리됩니다).

감사로그 목록 (그림 17)

감사로그 상세 — 변경 전/후 비교 (그림 18)

목록 화면에서는 테이블/작업유형/회사/프로젝트/작업자/작업일시로 필터링해 원하는 변경 건을 찾고, 상세 화면에서 변경 전/후 값을 비교합니다(`api_secret` 처럼 민감한 값은 `***` 로 마스킹되어 기록됩니다).

8. 게임서버 개발자를 위한 연동 안내

이 장은 Coupon Platform과 여러분의 게임서버를 실제로 연결하는 **백엔드 개발자**를 위한 내용입니다. 관리 콘솔 사용자 인증(로그인 ID/비밀번호, JWT)과는 완전히 별개의 인증 방식을 쓴다는 점에 유의합니다 — 게임서버는 5.3에서 발급받은 **API Key + API Secret**으로만 인증합니다.

8.1 공통 API 계약

Coupon Platform이 게임서버를 위해 노출하는 API(쿠폰 사용 관련)는 아래 세 개입니다. 전부 `POST` 이고 `/v1` 버전 접두어가 붙습니다.

<code>POST /v1/coupons/{code}/reserve</code>	쿠폰 사용(소모 확정)
<code>POST /v1/coupons/{code}/confirm</code>	지급 결과 통보
<code>POST /v1/coupons/unconfirmed</code>	<code>confirm</code> 이 안 온 소모 건 조회(재처리용)

응답은 아래 봉투(envelope) 형식을 공통으로 따릅니다.

```
{
  "result": 0,
  "data": { "...": "..." }
}
```

- result 가 0 이면 성공, 그 외 값이면 실패입니다. HTTP Status Code와 Result Code를 함께 확인해야 합니다(HTTP만으로는 실패 사유를 구분할 수 없는 경우가 있습니다).
- 성공 시 항상 result: 0 과 함께 HTTP 200(또는 202)을 반환하며, 비즈니스 오류를 HTTP 200으로 감춰서 반환하지 않습니다.
- 요청 파라미터는 GET 쿼리스트링이 아니라 **항상 JSON 바디**로 전달합니다(조회 API인 /coupons/unconfirmed 도 마찬가지입니다) — URL에 유저 식별자 같은 값이 남아 접근 로그 등에 그대로 찍히는 것을 피하기 위한 설계입니다.

HTTP Status Code는 Result Code의 범위에 따라 아래처럼 매핑됩니다.

HTTP Status	Result Code 범위	의미
200 / 202	0	정상 처리(202는 비동기 접수)
400	30000~39999	입력값/쿠폰 사용 제약 오류
401	10000~19999	인증 실패
404	31000~31999	대상 없음(Result Code 범위 안에서 세분화)
429	40000~49999	요청 제한 초과
500	50000~59999	서버 내부 오류

31000번대는 30000~39999 범위에 포함되지만, HTTP Status는 400이 아니라 **404가 우선 적용됩니다**. 즉 표는 위에서 아래로 갈수록 더 넓은 범위가 아니라, **좁은 범위(31000번대)가 넓은 범위(30000번대)보다 우선한다**는 뜻으로 읽어야 합니다.

게임서버 연동 시 자주 마주치는 오류 코드는 아래와 같습니다.

Result Code	HTTP	의미
30001	400	필수 입력값 누락(game_user_id 등)
31005	404	쿠폰 코드 없음(오타/존재하지 않는 코드)
31006	404	소모 기록 없음(reserve 없이 confirm 시도)
33001	400	코드 이미 소모됨/중지됨
33002	400	캠페인 사용 불가(비활성/기간 외/오픈 수량 소진)
33003	400	이 유저의 사용 한도 초과
40001	429	요청 제한 초과(아래 참고)

인증 단계 자체가 실패하면 위 표에 없더라도 아래 코드가 공통으로 반환될 수 있습니다(8.2 참고).

Result Code	HTTP	의미
-------------	------	----

10010	401	API Key 없음/유효하지 않음
10011	401	서명(Signature) 불일치
10012	401	필수 인증 헤더 누락/형식 오류
10013	401	Timestamp 허용범위 초과
10014	401	프로젝트 사용중지
10015	401	Nonce 재사용 감지(재전송 의심)

요청 제한(Rate Limit): 프로젝트(API Key) 단위로 순간 최대 처리량과 초당 처리율이 정해져 있습니다. 이는 오남용 방지가 아니라 특정 게임서버의 트래픽 폭주로부터 플랫폼 전체를 보호하기 위한 것이며, 정상적인 서비스 트래픽이라면 거의 마주칠 일이 없는 여유 있는 값으로 설정되어 있습니다. 초과 시 429와 함께 `Retry-After` 헤더(최소 대기 초)가 함께 옵니다.

8.2 S2S 인증 서명 만들기

Coupon Platform은 API Secret 원문을 요청에 절대 실지 않습니다 — 대신 매 요청마다 **HMAC-SHA256으로 서명한 값**을 실어 보냅니다. 아래 4개 헤더가 모든 요청에 필요합니다.

헤더	설명
X-API-Key	발급받은 API Key
X-API-Timestamp	요청 생성 시각, Unix Epoch 초 단위 정수 문자열
X-API-Nonce	요청마다 새로 생성하는 1회성 임의의 문자열(UUID 권장)
X-API-Signature	아래 규칙으로 계산한 서명(hex)

서명 대상 문자열(string to sign)은 다음과 같이 줄바꿈(`\n`)으로 이어 붙여 구성합니다.

```
stringToSign = HTTP_METHOD + "\n"
               + PATH (쿼리스트링 제외) + "\n"
               + RAW_QUERY_STRING (없으면 빈 문자열) + "\n"
               + X-API-Timestamp + "\n"
               + X-API-Nonce + "\n"
               + RAW_BODY (없으면 빈 문자열)

X-API-Signature = HMAC-SHA256(secret, stringToSign) // hex 인코딩
```

Node.js 참고 구현입니다.

```
const { createHmac, randomUUID } = require('crypto');

function buildS2sHeaders(apiKey, apiSecret, method, path, rawBody) {
  const timestamp = Math.floor(Date.now() / 1000).toString();
  const nonce = randomUUID();
  const stringToSign = [method, path, '', timestamp, nonce, rawBody].join('\n');
  const signature = createHmac('sha256', apiSecret).update(stringToSign).digest('hex');
```

```

return {
  'Content-Type': 'application/json',
  'X-API-Key': apiKey,
  'X-API-Timestamp': timestamp,
  'X-API-Nonce': nonce,
  'X-API-Signature': signature,
};
}

```

다른 언어/프레임워크를 쓰더라도 같은 로직(HMAC-SHA256, 줄바꿈 구분 문자열, hex 인코딩)만 그대로 옮기면 됩니다. 실수하기 쉬운 지점 세 가지를 짚어드립니다.

RAW_BODY는 "보낼 그 문자열 그대로" 서명해야 합니다. JSON을 만든 뒤 서명용으로 다시 직렬화(re-serialize)하면 키 순서나 공백 차이로 서버가 계산한 서명과 어긋나 정상 요청도 거부될 수 있습니다. 서명에 쓴 문자열과 실제로 전송하는 바디 문자열이 완전히 동일해야 합니다.

`{code}` 경로에 URL 인코딩이 필요한 문자가 있다면, 서명에 넣는 PATH와 실제로 요청을 보내는 URL의 경로가 정확히 같은 문자열이어야 합니다. 서버는 받은 경로 문자열을 그대로 서명 검증에 쓸 뿐 별도로 디코딩하지 않습니다.

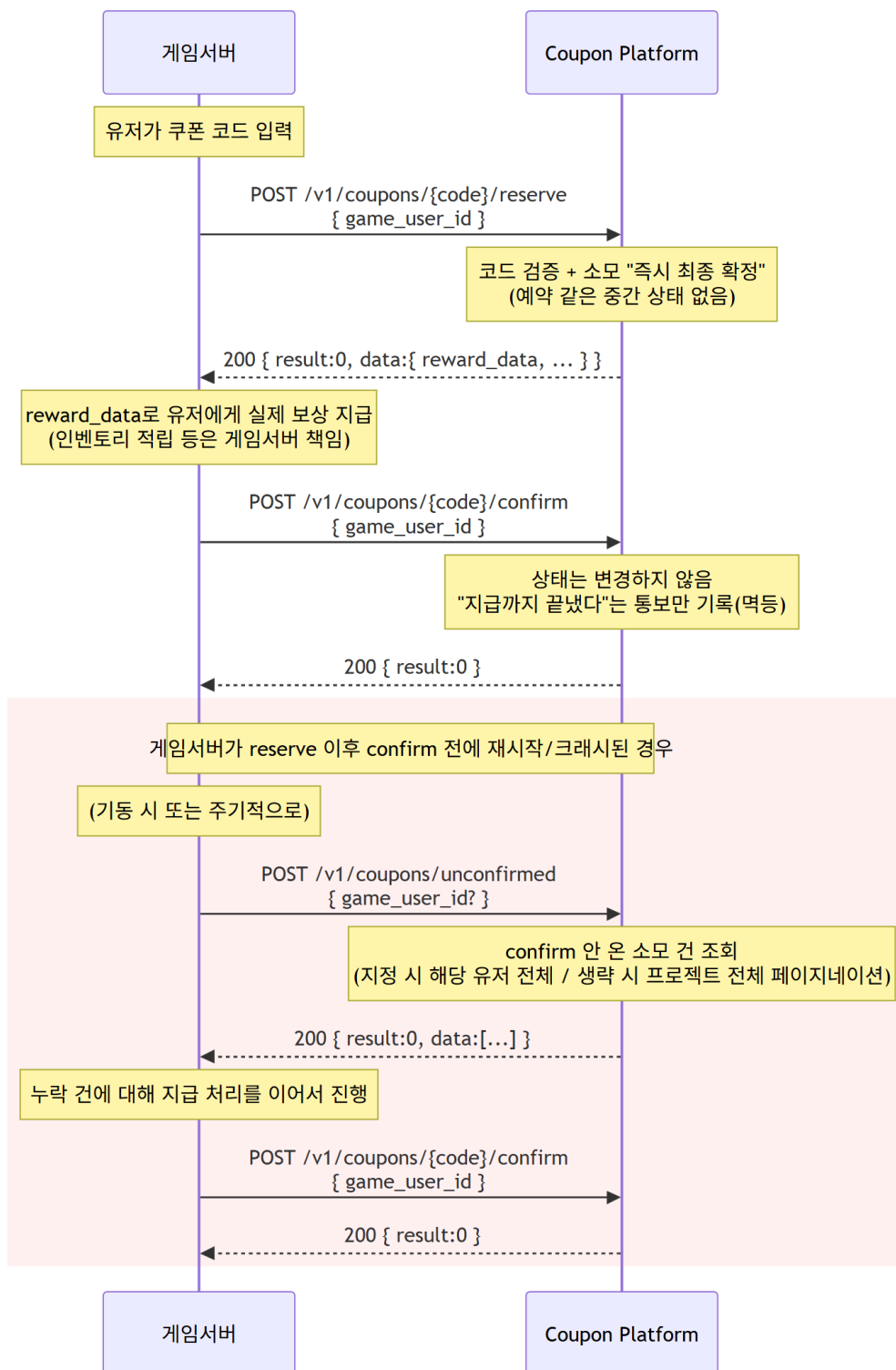
`X-API-Timestamp` 는 서버와 시계가 크게 어긋나면 거부됩니다(허용 오차를 벗어나면 인증 실패). 게임서버가 NTP 등으로 시각 동기화를 유지하고 있는지 확인하세요.

시각 동기화 오차가 실제로 발생하는 경우: 게임서버가 이 오차로 인증에 계속 실패한다면 서버 시각을 먼저 점검하고, 필요하면 운영 담당자에게 허용 오차 값 조정이 가능한지 문의해 주세요.

동일한 로직을 그대로 구현한 재사용 가능 참고 SDK(외부 의존성 없이 Node.js 내장 기능만 사용)를 [부록 A](#)에 전체 소스로 실어뒀습니다 — 처음부터 직접 구현하는 대신 참고하시거나 그대로 가져다 쓰셔도 됩니다.

8.3 쿠폰 사용 API (reserve / confirm / 미컨슈م 조회)

정상 흐름과, 게임서버 장애 시 복구 흐름을 시퀀스로 정리하면 다음과 같습니다.



Reserve — 쿠폰 사용(소모 확정)

```
POST /v1/coupons/{code}/reserve
```

```
{ "game_user_id": "player_1001" }
```

호출 즉시 소모가 최종 확정됩니다. 예약 같은 중간 상태 없이, 성공하면 그 자리에서 코드가 소모 완료 처리됩니다. 응답에 `reward_data` (캠페인 등록 시 입력한 보상 내용)가 그대로 포함되므로, 게임서버는 이 응답만으로 즉시 지급 처리를 진행할 수 있습니다.

```
{
  "result": 0,
  "data": {
    "coupon_code_usage_id": 9000,
    "coupon_campaign_id": 100,
    "code_value": "23A4-B7C9-DEF2",
    "game_user_id": "player_1001",
    "reward_data": { "item_id": 5001, "qty": 3 },
    "created_at": "2026-07-27 10:00:00"
  }
}
```

Confirm — 지급 결과 통보

```
POST /v1/coupons/{code}/confirm
{ "game_user_id": "player_1001" }
```

reserve로 이미 소모는 끝났으므로, confirm은 코드/캠페인의 어떤 상태도 바꾸지 않습니다. "지금까지 완료했다"는 결과를 통보하는 용도일 뿐입니다. 여러 번 호출해도 무해합니다(먹등).

미컨슈م 조회 — confirm이 안 온 건 재처리

```
POST /v1/coupons/unconfirmed
{ "game_user_id": "player_1001" }
```

reserve는 성공했는데 confirm을 보내지 못한 채로 게임서버가 재시작되거나 크래시된 경우를 위한 조회 API입니다. `game_user_id` 를 지정하면 그 유저의 미통보 건 전체를(페이지네이션 없이), 생략하면 프로젝트 전체를 오래된 순으로 페이지네이션 조회합니다. 게임서버가 기동 시 또는 주기적으로 이 API를 호출해, 남아있는 건에 대해 지급 처리를 이어서 진행하고 confirm을 다시 보내는 방식으로 사용합니다.

8.4 재시도와 먹등성 — 반드시 알아야 할 함정

네트워크 타임아웃이나 장애로 응답을 못 받았을 때 **reserve**를 그대로 재시도해도 안전한지는 캠페인 설정에 따라 다릅니다.

- **유저당 사용한도(`use_limit_per_user`)가 1인 캠페인(RANDOM/FIXED 공통):** 동일한 코드+동일한 `game_user_id` 로 재시도하면 새로 소모하지 않고 **최초 성공 때와 완전히 동일한 응답**을 그대로 다시 돌려줍니다(먹등). 안전하게 재시도해도 됩니다.
- **유저당 사용한도가 1보다 큰 캠페인(FIXED만 해당):** "이게 재시도인지, 유저가 정말 한 번 더 쓴 것인지" 서버가 구분할 방법이 없어 이 먹등 처리가 적용되지 않습니다. 재시도도 그냥 새로운 소모 1건으로 카운트됩니다 — 한도 안이면 새로 성공하고, 한도를 넘으면 33003으로 실패합니다. **이런 캠페인에서 응답을 못 받**

은 요청을 무작정 재시도하면 유저의 한도를 의도치 않게 두 번 소모시킬 수 있습니다. 재시도 전에 8.3의 미션숨 조회로 실제 소모 여부를 먼저 확인하는 것을 권장합니다.

자주 헛갈리는 질문: "A유저가 어떤 코드로 사용요청을 보냈고, 1초 뒤 완전히 동일한 요청이 다시 들어오면 결과는 어떻게 되나요?" — 위 기준으로, 그 캠페인의 유저당 사용한도가 1이면 **오류 없이 최초와 동일한 성공 응답이 돌아옵니다**(중복 소모가 아닙니다). 한도가 1보다 큰 **FIXED** 캠페인이면 **정상적인 두 번째 소모로 처리됩니다**(한도 내라면 성공, 초과라면 33003).

nonce는 재시도할 때도 매번 새로 생성해야 합니다. nonce는 재전송 공격을 막기 위한 값이라, 같은 nonce로 두 번째 요청을 보내면(설령 재시도 의도였더라도) 재전송으로 간주되어 그 요청 자체가 인증 단계에서 거부됩니다(10015). 즉 "같은 요청을 그대로 다시 보내면 안전하게 먹등 처리된다"는 것은 **비즈니스 로직(코드+유저 조합) 수준의 이야기**이고, HTTP 요청 자체는 매번 새로운 timestamp/nonce/서명으로 새로 만들어 보내야 합니다.

8.5 연동 절차 요약 (체크리스트)

1. Coupon Platform 운영 담당자에게 회사/프로젝트 생성을 요청하거나, 직접 SUPER_ADMIN 권한으로 프로젝트를 생성합니다(5.1).
2. 프로젝트 상세 화면에서 **API Key/Secret**을 확인하고, 게임서버 환경변수 등에 안전하게 보관합니다(5.3).
3. 8.2의 서명 규칙대로 요청 헤더를 만드는 로직을 게임서버에 구현합니다(부록 A의 참고 SDK 재사용 가능).
4. 관리 콘솔에서 캠페인을 등록하고 쿠폰 코드를 발급합니다(6.1~6.2). 유저에게 오픈하기 전, `usable_qty` 가 열려 있고 캠페인 상태가 "활성"인지 확인합니다.
5. 유저가 코드를 입력하는 시점에 `POST /v1/coupons/{code}/reserve` 를 호출하고, 응답의 `reward_data` 로 지급 처리 후 `POST /v1/coupons/{code}/confirm` 을 호출합니다.
6. 게임서버 기동 시(또는 주기적으로) `POST /v1/coupons/unconfirmed` 로 `confirm` 누락 건을 조회해 재처리합니다.
7. 운영 중 실패 사례를 확인해야 하면 관리 콘솔의 "쿠폰 사용 로그" 화면(6.3)에서 캠페인/유저/코드값/기간으로 필터링해 조회합니다.

9. 자주 묻는 질문

Q. 회원가입 화면에서 회사코드/프로젝트코드를 모르겠어요. 운영 담당자(SUPER_ADMIN)에게 문의해 코드를 받아야 합니다. 로그인 전 화면이라 목록이 공개되지 않습니다.

Q. 로그인이 안 됩니다. 계정 상태(가입승인대기/반려/사용중지)에 따라 각각 다른 안내 메시지가 표시됩니다. 승인 대기 상태라면 담당자의 승인을 기다려야 합니다.

Q. 가입 승인은 됐는데 캠페인 목록이 비어 있어요. 프로젝트 단위 권한(DEVELOPER/MANAGER/OPERATOR)이 아직 배정되지 않은 상태일 수 있습니다(2.1 참고). SUPER_ADMIN에게 권한 배정을 요청해야 합니다.

Q. 캠페인을 등록/수정했는데 반영이 안 됩니다. OPERATOR 권한으로 등록/수정하면 승인요청 상태로만 전환됩니다(6.1 참고). MANAGER 이상의 승인이 필요합니다.

Q. reserve 호출이 계속 33002로 실패합니다. 캠페인 상태가 "활성"이 아니거나, 사용기간 밖이거나, `usable_qty` (오픈 수량)가 소진됐을 가능성이 높습니다. 캠페인 상세 화면에서 상태/기간/수량을 먼저 확인하세요.

Q. reserve/confirm 호출이 계속 인증 실패(401)로 떨어집니다. 서명 계산 로직(8.2)을 다시 점검하세요 — RAW_BODY 재직렬화 여부, 경로 인코딩 일치 여부, 서버와의 시각 동기화 순으로 확인하는 것이 가장 빠릅니다. X-API-Key 자체가 틀렸다면 10010, 서명 불일치는 10011, 헤더 누락은 10012, 시각 오차는 10013으로 각각 다르게 응답되니 어떤 코드인지부터 확인하세요.

Q. API Secret을 재발급했더니 기존 게임서버 인스턴스 일부가 인증 실패로 떨어집니다. 재발급 직후에는 유예기간 동안 이전 Secret도 함께 유효하므로 정상적으로는 즉시 끊기지 않습니다(5.3 참고). 유예기간이 이미 지났거나, 애초

에 새 Secret을 게임서버 설정에 반영하지 않았는지 확인해 보세요.

10. 문의

이 문서에서 다루지 않는 내용이나 오류가 발생하면 화면 하단 푸터에 표시된 문의처로 연락하거나, Coupon Platform 운영 담당자에게 문의해 주세요.

부록 A. CouponS2sClient 참고 SDK 전체 소스

8.2절에서 안내한 S2S 인증 서명 로직을 그대로 구현한 TypeScript 참고 SDK 전체 소스입니다. 외부 의존성 없이 Node.js 22 LTS 내장 `crypto` (HMAC 서명)와 전역 `fetch` (HTTP 호출)만 사용하며, 처음부터 직접 구현하는 대신 그대로 가져다 쓰셔도 됩니다.

```
import { createHmac, randomUUID } from 'crypto';

/**
 * coupon_platform S2S(Server-to-Server) 쿠폰 사용 API 연동 SDK - 입점사(게임사)가 그대로
 * 가져다 쓸 수 있는 독립 산출물이다.
 *
 * - 외부 의존성 0개: Node.js 22 LTS 내장 `crypto`(HMAC 서명)와 전역 `fetch`(HTTP 호출)만 쓴다.
 * - 입력은 항상 평문 `apiKey`/`apiSecret`이다 - 이 파일 자체에는 복호화 로직이 없다.
 * - 서명 규칙은 8.2절, 실제 서버의 서명 검증 로직과 동일하다.
 */

export interface CouponS2sClientOptions {
  /** 예: https://coupon-api.example.com (끝에 슬래시 없이) */
  baseUrl: string;
  apiKey: string;
  /** 평문 API Secret - 발급/재발급 API 응답으로 1회 노출된 값을 그대로 사용한다. */
  apiSecret: string;
}

export interface ReserveResult {
  coupon_code_usage_id: number;
  coupon_campaign_id: number;
  code_value: string;
  game_user_id: string;
  reward_data: unknown;
  created_at: string;
}

export interface ConfirmResult {
  coupon_code_usage_id: number;
  confirmed_at: string;
}

export interface UnconfirmedItem {
  code_value: string;
  game_user_id: string;
}
```

```

    coupon_campaign_id: number;
    reward_data: unknown;
    created_at: string;
}

export interface UnconfirmedResultSingleUser {
    items: UnconfirmedItem[];
}

export interface UnconfirmedResultPaginated {
    page: number;
    page_size: number;
    total_count: number;
    items: UnconfirmedItem[];
}

export type UnconfirmedResult =
    | UnconfirmedResultSingleUser
    | UnconfirmedResultPaginated;

export interface GetUnconfirmedParams {
    /** 지정 시 특정유저 조회 모드(페이지네이션 없음). 미지정 시 전체유저 조회 모드(page/pageSize 필
    수). */
    gameId?: string;
    campaignId?: number;
    page?: number;
    pageSize?: number;
}

/** API가 `{result != 0}`을 반환했을 때 던지는 에러 - 입점사가 `resultCode`로 분기하기 위함. */
export class CouponApiError extends Error {
    constructor(
        public readonly resultCode: number,
        public readonly httpStatus: number,
        message: string,
    ) {
        super(message);
        this.name = 'CouponApiError';
    }
}

export class CouponS2sClient {
    private readonly baseUrl: string;
    private readonly apiKey: string;
    private readonly apiSecret: string;

    constructor(options: CouponS2sClientOptions) {
        this.baseUrl = options.baseUrl.replace(/W/+$/, '');
        this.apiKey = options.apiKey;
        this.apiSecret = options.apiSecret;
    }
}

```

```

/**
 * `POST /v1/coupons/{code}/reserve` - 8.3절. 성공 시 즉시 최종 소모 확정이다(예약 중간
 * 상태 없음) - 응답을 못 받아 재시도할 때의 동작이 캠페인 설정에 따라 달라지므로 반드시
 * 알아두어야 한다(8.4절 참고):
 *
 * - `use_limit_per_user`가 1인 캠페인(RANDOM/FIXED 공통): 동일한 `codeValue`+`gameUserId`로
 *   재시도하면 새로 소모하지 않고 최초 성공 때와 완전히 동일한 응답을 그대로 재반환한다
 *   (먹등) - 네트워크 타임아웃 등으로 응답을 못 받았을 때 안전하게 재시도해도 된다.
 * - `use_limit_per_user`가 1보다 큰 캠페인(FIXED만 해당): "재시도"와 "정당한 반복 사용"을
 *   서버가 구분할 방법이 없어 먹등 처리가 적용되지 않는다 - 재시도도 그냥 새로운 소모 1건으로
 *   카운트된다(한도 안이면 새 `coupon_code_usage_id`로 성공, 한도를 넘으면 `33003`). 이런
 *   캠페인에서 응답을 못 받은 요청을 무작정 재시도하면 의도치 않게 사용자의 한도를 두 번
 *   소모시킬 수 있다 - 재시도 전에 `getUnconfirmed()`로 실제 소모 여부를 먼저 확인하는 것을
 *   권장한다.
 */
async reserve(codeValue: string, gameUserId: string): Promise<ReserveResult> {
  return this.request<ReserveResult>({
    '/v1/coupons/${encodeURIComponent(codeValue)}/reserve',
    { game_user_id: gameUserId },
  });
}

/** `POST /v1/coupons/{code}/confirm` - 8.3절 */
async confirm(codeValue: string, gameUserId: string): Promise<ConfirmResult> {
  return this.request<ConfirmResult>({
    '/v1/coupons/${encodeURIComponent(codeValue)}/confirm',
    { game_user_id: gameUserId },
  });
}

/**
 * `POST /v1/coupons/unconfirmed` - 8.3절. 조회 전용이지만 GET이 아니라 POST다 - GET
 * 쿼리스트링에 `game_user_id` 등을 실으면 웹서버/프록시/CDN 접근 로그에 그대로 남기 쉬워,
 * 나머지 두 엔드포인트와 동일하게 바디로 받도록 통일했다.
 */
async getUnconfirmed(params: GetUnconfirmedParams = {}): Promise<UnconfirmedResult> {
  const body: Record<string, unknown> = {};
  if (params.gameUserId !== undefined) body.game_user_id = params.gameUserId;
  if (params.campaignId !== undefined) body.campaign_id = params.campaignId;
  if (params.page !== undefined) body.page = params.page;
  if (params.pageSize !== undefined) body.page_size = params.pageSize;

  return this.request<UnconfirmedResult>('/v1/coupons/unconfirmed', body);
}

/**
 * 서명 생성 + HTTP 호출 + 응답 파싱을 한 곳에 모은 내부 헬퍼. 3개 엔드포인트 전부 POST+바디라
 * HTTP 메서드를 인자로 받지 않는다. `path`는 caller가 넘긴 문자열을 그대로 서명(`stringToSign`)과
 * 실제 fetch URL 양쪽에 동일하게 쓴다 - 서버(Express `request.path`)는 퍼센트 디코딩을
 * 하지 않으므로(`req.path`는 인코딩된 문자열 그대로다), 클라이언트가 보낸 그대로의 문자열이
 * 서명 검증 기준과 일치해야 한다. reserve/confirm이 이 메서드에 넘기는 path는 이미
 * `encodeURIComponent`를 거친 상태인데, 코드값이 URL-safe 문자(nanoid 알파벳/하이픈)로만

```

```

* 구성되면 인코딩 전후가 동일해 실질적으로 문제가 없다.
*/
private async request<T>(path: string, body: Record<string, unknown>): Promise<T> {
    const bodyString = JSON.stringify(body);
    const timestamp = Math.floor(Date.now() / 1000).toString();
    const nonce = randomUUID();
    const stringToSign = ['POST', path, '', timestamp, nonce, bodyString].join('Wn');
    const signature = createHmac('sha256', this.apiSecret)
        .update(stringToSign)
        .digest('hex');

    const response = await fetch(`${this.baseUrl}${path}`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'X-API-Key': this.apiKey,
            'X-API-Timestamp': timestamp,
            'X-API-Nonce': nonce,
            'X-API-Signature': signature,
        },
        body: bodyString,
    });

    const json = (await response.json()) as { result: number; data?: T; message?: string };

    if (json.result !== 0) {
        throw new CouponApiError(
            json.result,
            response.status,
            json.message ?? `coupon API error (result=${json.result})`,
        );
    }

    return json.data as T;
}
}

```

이 문서는 Coupon Platform 프로젝트의 GitHub README와 이용 가이드를 결합해 작성한 포트폴리오용 문서입니다. 저장소: https://github.com/trisakion0500/coupon_platform